WEB DEVELOPMENT I · THE CLIENT · U-TAD · WEBI ARENA

M2 - ASYNC ODYSSEY

U2 · JAVASCRIPT AVANZADO Y HERRAMIENTAS DEL FRONTEND

«El event loop no espera a nadie.»

SE ABRE	ENTREGA	DEFENSA ESCRITA
Miércoles 30 de septiembre	**Martes 13 de octubre, 23:59**	**Miércoles 14 de octubre, en clase**

LA MISIÓN

Crea una aplicación (con Vite) que consuma una API pública real (PokeAPI, TheMealDB, OpenLibrary…) usando `fetch` + `async/await`, módulos ES y métodos de array. Renderiza los datos en el DOM con estados de carga y error.

OBJETIVOS

- `fetch` con `async/await` y manejo de errores (`try/catch`, `response.ok`)
- `map` / `filter` / `reduce` para transformar los datos antes de pintarlos
- Módulos ES: separar API, lógica y render
- Proyecto arrancado con Vite y `package.json` coherente

> ★ RETO BONUS
> Añade una caché en `localStorage` para no repetir peticiones ya hechas.

CÓMO SE PUNTÚA - 100 PTS

La rúbrica es pública: sabes exactamente cómo se puntúa antes de empezar. 70 puntos los evalúa la IA con esta rúbrica y 30 salen de comprobaciones automáticas del repositorio. Solo cuenta tu mejor intento: puedes volver a entregar cuantas veces quieras hasta la fecha límite.

CRITERIO	QUÉ SE MIRA	PUNTOS
Asincronía	async/await bien usado, estados de carga y error visibles para el usuario	20
Transformación de datos	Uso idiomático de métodos de array (nada de bucles innecesarios)	15
Módulos y estructura	Separación clara en módulos ES con responsabilidades definidas	15
Robustez	La app no explota con respuestas vacías, errores de red o datos raros	10
Originalidad	Elección de API y presentación con personalidad propia	10
Rúbrica evaluada por IA		70
README útil	README.md con qué es el proyecto, cómo arrancarlo y qué decisiones tomaste	4
Declaración de IA	La sección «Uso de IA» en el README, con sustancia (ver abajo)	4
Repositorio limpio	Sin `node_modules`, `dist` ni `build` commiteados: usa un `.gitignore`	5

CRITERIO	QUÉ SE MIRA	PUNTOS
Historia de commits	Al menos 5 commits: que se vea evolucionar el trabajo, no un volcado final	6
Tecnología de la misión	Proyecto Vite con su `package.json` y código en `.js` o `.ts`	8
Estructura del proyecto	El código repartido en varios archivos fuente (al menos 3)	3
Checks automáticos		30
TOTAL		**100**

EL README Y LA IA

> REGLA DE ORO
> Puedes usar IA para todo, pero todo lo que entregues lo tienes que poder defender línea a línea: la nota no es por escribir código, es por entenderlo.

El README de tu entrega debe incluir dos secciones:

«Uso de IA» — Qué partes hiciste con IA y con qué herramienta, 1–2 prompts reales relevantes, cómo verificaste lo generado y qué escribiste a mano. Si no usaste IA, dilo y listo.

«Autopsia» — Las 2 decisiones más discutibles de tu código y qué alternativa descartaste en cada una.

Usar más o menos IA no puntúa distinto; lo que se exige es declararlo y poder defenderlo.

Puedes entregar el código en castellano o en inglés: el idioma no influye en la nota. Te recomendamos el de tu clase y, sobre todo, ser coherente dentro del mismo proyecto —se lee mucho mejor—, pero ni una cosa ni la otra puntúan.

CÓMO SE ENTREGA

1. **Repositorio público en GitHub.** Vale un repositorio entero o una carpeta dentro de uno (`https://github.com/usuario/repo/tree/main/carpeta`). Haz commits a medida que avanzas.
2. **En WebI Arena**, pestaña **Entregar**: elige la **M2 · Async Odyssey** y pega la URL. Ahí recibirás tu puntuación y el feedback detallado cuando el profesor lance la evaluación.
3. **En esta tarea de Blackboard**: pega la misma URL que entregaste en la Arena antes de la fecha límite. Es la entrega formal de la asignatura.

DEFENSA

La nota es **provisional hasta la defensa**. El miércoles 14 de octubre, en clase, responderás por escrito a unas preguntas sobre **tu** código, sin tenerlo delante: qué hace, cómo lo organizaste y por qué tomaste tus decisiones. Si alguna respuesta necesita aclararse, se te citará para una defensa en persona. Si en la defensa demuestras menos del 50 %, la nota de la misión se multiplica por ese porcentaje.